Lab 5: Static Code Analysis

Pylint, Bandit and Flake8 on restaurant_order_system.py

Field	Details
Name	Sriviswaja M
SRN	PES1UG24AM288
Section	E
Assigned repository	SETAPESU26/L5_49_restaurant_order_system
Submission repository	github.com/sriviswaja/SE-Lab-AM288 (Lab 5 folder)

1. Objective

To improve the quality, security and style of an assigned Python program (a restaurant order billing and kitchen ticket module) by running three static analysis tools, documenting the issues they report, fixing them, and verifying the fixes by re-running the tools.

- **Pylint:** code quality and logic issues (naming, unused code, bad practices).
- **Bandit:** security vulnerabilities (dangerous functions, insecure patterns).
- **Flake8:** PEP 8 style and formatting.

2. Results Summary

Tool	Before	After
Pylint	28 messages, score 4.81 / 10	0 messages, score 10.00 / 10
Bandit	5 issues (2 High, 1 Medium, 2 Low)	No issues identified
Flake8	9 violations	0 violations

The program's behaviour was preserved: before and after the changes it reports the same pending status, the same discounted bill (40.5 for T1 at 10% off) and the same pending-table count.

3. Issues Identified and Fixes

Line numbers refer to the original file. Where multiple tools flagged the same problem, they are listed together.

#	Issue	Tool / Code	Line(s)	Description	Fix Approach
1	eval() on user input	Bandit B307 (Medium), Pylint W0123	42	promo_expr string executed as Python code; arbitrary code execution risk	Removed eval; function now takes a numeric discount_rate and computes base_bill * (1 - discount_rate)
2	Shell command injection	Bandit B605 (High)	58	os.system() built a shell command by concatenating filename	Replaced with print(); no shell is spawned
3	Weak hash (MD5)	Bandit B324 (High)	47	MD5 is cryptographically broken (collisions can be forged)	Switched to hashlib.sha256
4	Insecure random for PIN	Bandit B311 (Low)	51	random module is predictable; unsuitable for PINs	Used 1000 + secrets.randbelow(9000)

#	Issue	Tool / Code	Line(s)	Description	Fix Approach
5	try / except / pass	Bandit B110 (Low), Pylint W0702, Flake8 E722	25-27	Bare except silently swallowed every error	Catch KeyError only and print a message
6	Hardcoded secret	Manual review (not flagged)	71, 80	POS token hardcoded in source and printed to console	Read from POS_TERMINAL_KEY environment variable; print only whether it is set
7	Mutable default argument	Pylint W0102	15	order_log=[] is shared across all calls	Default to None and create a new list inside the function
8	Non-snake_case names	Pylint C0103	15, 23, 30, 40	camelCase function names violate PEP 8	Renamed to place_table_order, void_table_order, check_table_status, calculate_discounted_bill
9	Missing docstrings	Pylint C0116	15-61	Functions had no documentation	Added a one-line docstring to every function
10	Comparison to None with ==	Pylint C0121, Flake8 E711	32	== None can be overridden; identity check is correct	Changed to is None
11	Redundant if/else return	Pylint R1703, R1705	34-37	if/else returning True/False	return not orders[table_id] ["is_settled"]
12	Unused variables	Pylint W0612, Flake8 F841	31, 41, 63	service_tax, t unused; base_bill unused once eval was removed	Removed service_tax; iterate with .values(); base_bill now used
13	File not closed safely / no encoding	Pylint R1732, W1514	55	open() without with or encoding; file leaks on exceptions	with open(filename, "w", encoding="utf-8") as f:
14	Shadowing global name	Pylint W0621	61	Parameter orders shadowed the global orders dict	Renamed parameter to order_book
15	Unused import	Pylint W0611, Flake8 F401	6	math imported but never used	Removed import
16	Old-style % formatting	Pylint C0209	19	% formatting is less readable	Converted to an f-string
17	Line too long	Pylint C0301, Flake8 E501	18, 19, 75	Lines exceeded 79 (PEP 8) / 100 (Pylint) characters	Split the order dict across lines; renames shortened the others
18	Missing final newline	Pylint C0304, Flake8 W292	83	File did not end with a newline	Added trailing newline
19	Module runs on import	Manual review	83	main() called at module level, so importing the file runs it	Wrapped in if __name__ == "__main__":

4. Commits

The fixes were committed in three stages, one per tool, so each commit's diff shows only that tool's fixes.

Commit	Tool	Changes
`ef09a31`	Bandit	Removed eval and os.system; MD5 to SHA-256; random to secrets; bare except narrowed to KeyError; secret moved to environment variable
`9f08fd0`	Pylint	snake_case names, docstrings, mutable default, f-string, is None, simplified return, removed unused code, with open(..., encoding), renamed shadowing parameter, __main__ guard
`5834f74`	Flake8	Split the long order dictionary to fit the 79-character PEP 8 limit

Additional commits contain the original code (26a745d) and the before / after reports.

Key fix: removing eval()

Before:

```
net_total = eval(promo_expr)   # called with "base_bill * 0.90"
```

After:

```
return base_bill * (1 - discount_rate)   # called with 0.10
```

eval() executes any string as Python, so a malicious "promo code" such as __import__('os').system(...) could run arbitrary commands. Accepting a number instead of an expression removes the attack surface entirely while computing the same result.

5. Reflection

Q1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest were the mechanical style and quality issues: the unused import, the missing final newline, == None to is None, adding docstrings and splitting long lines. Each has one obvious correct fix and cannot change program behaviour.

The hardest was the eval() call. Removing it was not just deleting a line; it meant changing the function's interface from "pass an expression string" to "pass a discount rate", updating the caller, and checking that the output (40.5) stayed the same. Renaming the camelCase functions also needed care, because every call site had to be updated consistently. The os.system call required deciding what the line was actually for (a status message) and replacing it with something safe that preserved that intent.

Q2. Did the static analysis tools report any false positives? If so, describe one example.

Bandit's B311 warning on random.randint is context-dependent. random is perfectly fine for simulations or games, and Bandit flags every use regardless of purpose, so in many programs this would be a false positive. Here it was a true positive because the value is a POS PIN, which is a security credential, so it was fixed with the secrets module.

The reverse also happened: a false negative. The hardcoded POS_TERMINAL_SECRET_TOKEN_2026 was not flagged by Bandit, because its hardcoded-password check matches variable names containing words like "password" or "token", and the variable was named pos_terminal_key. It was the most realistic security bug in the file, and it was found only by manual review. Static analysis tools are pattern matchers and cannot replace reading the code.

Q3. How would you integrate static analysis tools into your actual software development workflow?

- **Editor:** enable Pylint/Flake8 in VS Code so issues are flagged while typing, when they are cheapest to fix.
- **Pre-commit hooks:** run Flake8 and Bandit via the pre-commit framework so code that fails cannot be committed.
- **CI pipeline:** run all three tools in GitHub Actions on every push and pull request, failing the build on any High-severity Bandit issue or a Pylint score below a threshold (e.g. 9.5).
- **Shared configuration:** commit .pylintrc / setup.cfg so the whole team uses the same rules and line length.

Q4. What tangible improvements did you observe in code quality, readability, or potential robustness after applying the fixes?

- **Security:** removed two High-severity vulnerabilities (shell injection, weak hashing) and one Medium (arbitrary code execution through eval). PINs are now unpredictable and the secret is no longer in the source or the console output.
- **Robustness:** the mutable default argument bug, where log entries would leak between calls, is gone; the file is always closed even if writing fails; and voiding a missing table now gives a message instead of silently hiding every possible error.
- **Readability:** consistent snake_case names, docstrings on every function, a one-line return instead of an if/else, and f-strings make the code easier to read and maintain.
- **Reusability:** the __main__ guard means the module can be imported, for example by tests, without running the demo.
- **Measured:** Pylint rose from 4.81 to 10.00, and Bandit and Flake8 went from 5 and 9 issues to zero.

6. Conclusion

The three tools are complementary: Bandit found the security flaws, Pylint the logic and maintainability problems, and Flake8 the formatting issues, with some overlap (bare except, unused variables, == None). Together they took the file from a 4.81 Pylint score with five security issues to a clean pass on all three tools, without changing its behaviour. The hardcoded secret showed that these tools work best alongside, not instead of, manual code review.